Mandelbrot com MPI no Modelo Coordenador/Trabalhador

Trabalho 2 — Programação Paralela — Grupo 20 (conta LAD `cp32020`)
Gabriel Coelho Gabriel Cherubini
Escola Politécnica — PUCRS — 2026/2

1. Problema e mapeamento. Mantivemos o problema do T1: para cada pixel c de uma janela fixa do plano complexo, itera-se $z_{n+1} = z_n^2 + c$ até $|z| > 2$ ou 2000 iterações. O **saco de trabalho** é a imagem, uma matriz de inteiros que só o coordenador guarda; cada **vetor** é uma linha, e processá-lo é calcular as iterações dos seus pixels. As linhas são independentes, mas de custo muito desigual (Fig. 3, esq.): as centrais cruzam o conjunto e custam o máximo por pixel, as das bordas escapam logo. É o cenário em que distribuir sob demanda compensa.

2. Implementação SPMD. Um fonte gera as duas versões (`gcc` e `mpicc -DPARALELO`, ambas `-O2`). O rank 0 coordena e não calcula; os demais trabalham. O trabalhador toma a iniciativa com `PEDIDO`; recebe `TAREFA` com o índice da linha; devolve `RESULTADO` = [índice, pixels…], que já vale como novo pedido. O coordenador recebe com `MPI_ANY_SOURCE` (**ordem de chegada**) e usa o índice para gravar o vetor na sua linha, **mantendo a ordem do saco**; vazio o saco, responde `FIM`. Funciona para qualquer $p \geq 2$; `-k` agrupa k linhas por tarefa. O checksum (soma das iterações) é **idêntico** ao sequencial em todas as execuções (7 591 159 335). A instrumentação mede, em cada processo, o tempo calculando, bloqueado em `Recv/Send` e o instante de dispensa, e registra o início e fim de cada tarefa; é coletada após o tempo medido, sem afetá-lo.

3. Ambiente e metodologia. Dois nós exclusivos da atlantica (03 e 04), cada um com 2× Xeon E5520 de 2,27 GHz (4 núcleos cada: 8 núcleos e 16 threads HT por nó), Open MPI 4.1.1, gcc 9.4.0. $p = 2..32$ processos alternados entre os nós; até 16 há um por núcleo físico, com 32 usam-se as HT. Cada tempo é a mediana de 5 execuções (barras min–máx na Fig. 1); T_{seq} é o binário sequencial. Como p inclui o coordenador ocioso, o teto do speed-up é $p - 1$ e o da eficiência, $(p - 1)/p$.

4. Escalabilidade forte e fraca. Na forte (Tab. 1, Fig. 1), $S(16) = 13{,}92$ ($E = 87{,}0\%$, teto 93,8%) e $S(32) = 25{,}52$ ($E = 79{,}7\%$, teto 96,9%). Com $p = 2$ há um só trabalhador, e $S = 0{,}97$ só mede o custo do protocolo (3%). A fraca (Tab. 2) acompanha a forte ponto a ponto ($E(16) = 87{,}1\%$, $E(32) = 80{,}3\%$): as perdas não dependem do tamanho da imagem, e sim de p e de onde os processos rodam (seção 5). Contra o T1 em 1 nó: o OpenMP `dynamic,1` fez $S(8) = 6{,}90$ e $S(16) = 12{,}07$ com HT; o MPI faz $S(8) = 6{,}85$ com um processo a menos calculando e, com 16 processos num nó, $S = 12{,}78$.

5. Para onde vai a eficiência. Dividimos os recursos alocados, $p{\cdot}T$, em parcelas medidas (Fig. 2, Tab. 3): trabalho útil (T_{seq}, que é a própria eficiência); cálculo mais lento que no sequencial; o coordenador; comunicação; ociosidade final. A soma fecha em 100% (resíduo < 0,2%). Até $p = 8$ a perda é quase só o coordenador, que vale exatamente $1/p$ (50, 25, 12,5%); a comunicação fica entre 1,3 e 1,8% até $p = 16$. Duas parcelas crescem depois: com 16 (8 por nó, todos os núcleos ocupados) o mesmo cálculo fica 4,8% mais lento, e com 32, quando dois processos dividem cada núcleo por HT, 13,1%. É isso, e não a comunicação (4,1%), que separa os 79,7% do teto de 96,9%. O coordenador passa 95% do tempo ocioso esperando mensagens com $p = 32$: não é gargalo.

6. Balanceamento de carga. Com $k = 1$ o tempo calculando é quase igual entre os 31 trabalhadores (máx/média 1,007, CV 0,7%), embora cada um processe de 122 a 145 linhas: quem pega linhas caras do centro recebe menos (Fig. 3, dir.). Com um bloco fixo por trabalhador ($k = 133$, divisão estática) máx/média sobe para 2,79 e o tempo, 2,61× (4,62 s). O perfil de custo das linhas previa 4,17 s (o bloco mais caro custa 2,86× a média): o desbalanceamento vem do custo das linhas, não do MPI. O Gantt (Fig. 4) mostra todos terminando juntos no saco e, no estático, metade dos trabalhadores parada enquanto os blocos centrais terminam. Grão (Tab. 4): de $k = 1$ a 16 o tempo varia menos de 6% (melhor $k = 16$, 1,75 s); com $k = 64$ restam 64 tarefas para 31 trabalhadores, uns fazem 3 e outros 2, e o tempo sobe 1,34×.

7. Comunicação e rede. Cada mensagem leva uma linha (16 KB) e, no caso base, ≈11 ms de cálculo. Reduzindo `max_iter` com as mesmas mensagens (Fig. 5 esq.), com 746 µs de cálculo por mensagem $E = 51{,}5\%$; com 2,8 ms, 73%; acima de ≈5 ms a curva estabiliza em 78–82%. Esse patamar é o limite do HT da seção 5, e não da rede. Em 1×2 nós: $p = 8$ dá 82,7% num nó e 85,5% em dois, e $p = 16$ dá 79,9% num nó (com HT) e 87,0% em dois. Espalhar os processos pela rede foi **mais rápido** nos dois casos: o custo de dividir núcleo (HT) ou de lotar o nó, provavelmente pela perda do *turbo*, supera o da rede, que com mensagens tão espaçadas quase não aparece.

8. Uso de IA. Claude Code (Anthropic): implementação do protocolo, instrumentação, scripts de medição e análise, gráficos e esqueleto do relatório. O grupo definiu o mapeamento e a metodologia, executou e validou as medições e redigiu e revisou a análise.

9. Conclusão. O saco de trabalho chega a $S = 25{,}5$ com 32 processos em 2 nós ($E = 79{,}7\%$) e mantém a carga equilibrada (máx/média 1,007) num problema em que a divisão estática fica 2,6× mais lenta. A perda vem do coordenador ocioso ($1/p$) e, acima de 8 processos por nó, do cálculo mais lento em núcleos compartilhados; a comunicação custa só 1,3–4%. Com tarefas de milissegundos, espalhar por mais nós é melhor que usar HT.

Tab. 1 — Forte. 4096^2, 2000 it, $k = 1$, mediana de 5. $S = T_{seq}/T$, $E = S/p$.

p	Tempo (s)	S	E (%)	teto (%)
seq	45,137	—	—	—
2	46,652	0,97	48,4	50,0
4	15,330	2,94	73,6	75,0
8	6,589	6,85	85,6	87,5
16	3,243	13,92	87,0	93,8
32	1,769	25,52	79,7	96,9

Tab. 2 — Fraca. 1024^2 pixels por processo. $S = pT_{seq}/T$, $E = T_{seq}/T$.

p	Imagem	Tempo (s)	S	E (%)
seq	1024^2	2,823	1,00	100,0
2	1448^2	6,035	0,94	46,8
4	2048^2	3,859	2,93	73,2
8	2896^2	3,307	6,83	85,4
16	4096^2	3,243	13,93	87,1
32	5793^2	3,517	25,69	80,3

Tab. 3 — Decomposição de $p \cdot T$ (%), $k = 1$.

p	útil	cálc.+lento	coord.	comun.	ocioso fim
2	48,4	0,0	50,0	1,5	0,0
4	73,7	-0,0	25,0	1,3	0,0
8	85,7	0,3	12,5	1,5	0,0
16	87,1	4,8	6,2	1,8	0,0
32	79,7	13,1	3,1	4,1	-0,2

Tab. 4 — Grão, $p = 32$. $k = 133$: um bloco fixo por trabalhador (estático).

k	Tempo (s)	E (%)	ocup. máx/méd	vs melhor
1	1,769	79,7	1,01	1,01×
2	1,755	80,4	1,01	1,00×
4	1,857	76,0	1,01	1,06×
8	1,776	79,4	1,01	1,01×
16	1,753	80,5	1,02	1,00×
64	2,350	60,0	1,41	1,34×
133	4,619	30,5	2,79	2,64×

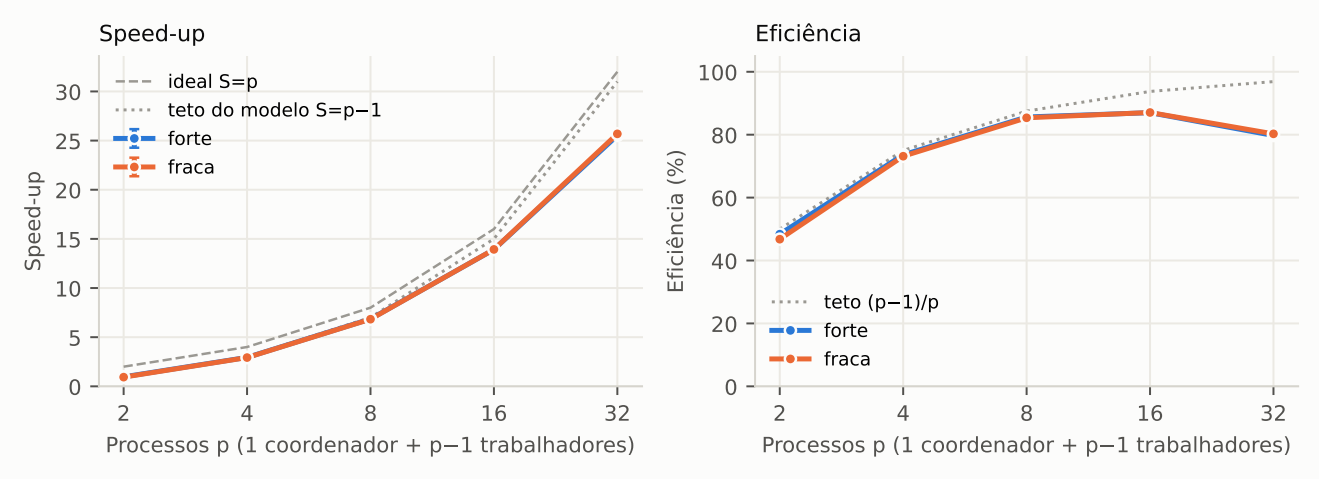


Fig. 1 — Speed-up (barras: mín–máx de 5 execuções) e eficiência. Tracejado: ideal $S = p$; pontilhado: teto do modelo, $p - 1$.

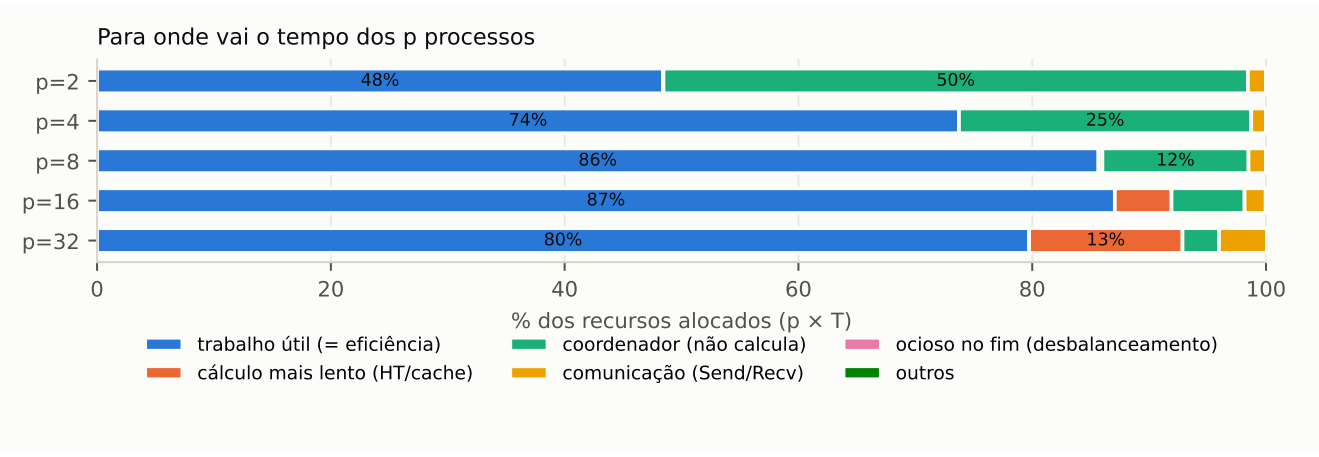


Fig. 2 — Recursos alocados ($p \times T$) divididos pelo que cada processo fez; a fatia azul é a eficiência.

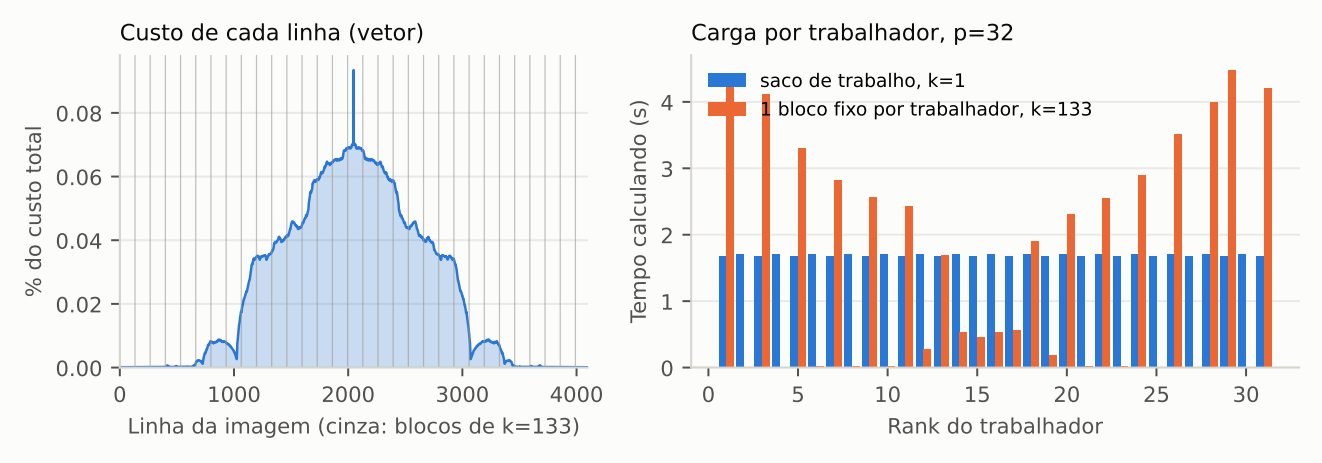


Fig. 3 — Esq.: custo de cada linha, com as fronteiras dos blocos estáticos. Dir.: tempo calculando de cada trabalhador, saco ($k = 1$) $\times$ bloco fixo ($k = 133$).

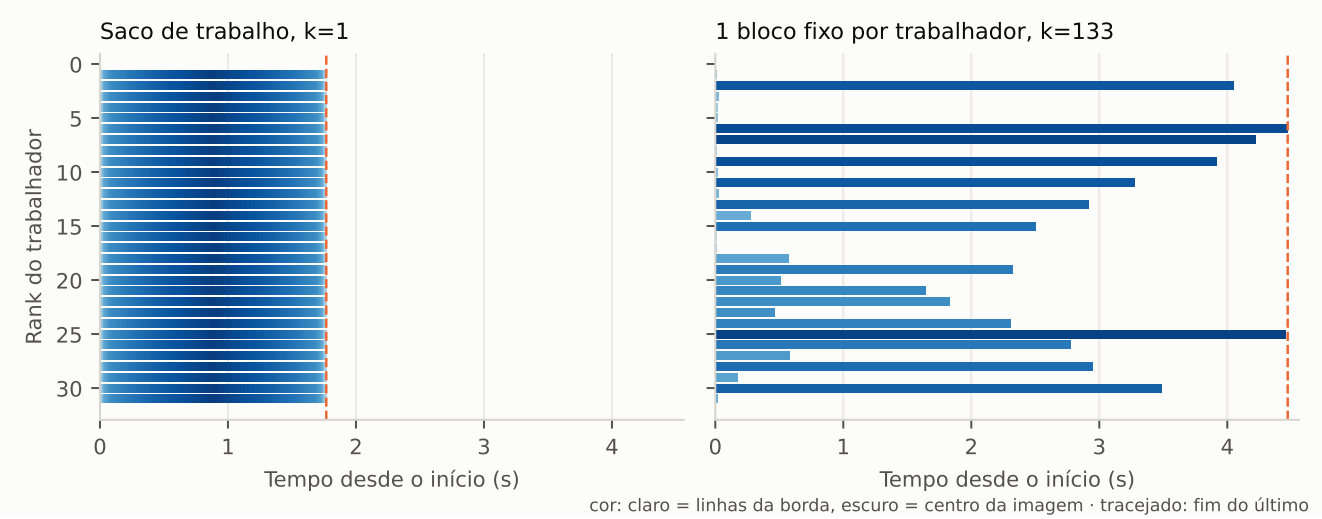


Fig. 4 — Linha do tempo das tarefas de cada trabalhador ($p = 32$).

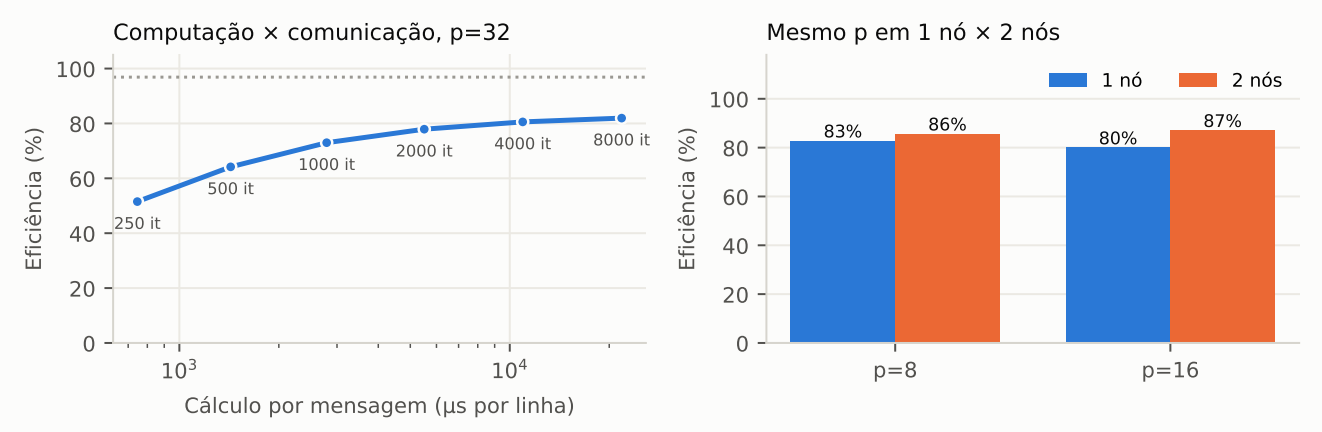


Fig. 5 — Esq.: eficiência conforme o cálculo por mensagem cresce (mesmo número de mensagens, `max_iter` variando). Dir.: o mesmo p em 1 e em 2 nós.

Código fonte — `mandelbrot.c`

```c
/*
 * Conjunto de Mandelbrot - Trabalho 2 de Computacao Paralela
 * MPI no modelo coordenador/trabalhador (saco de trabalho)
 * Grupo 20 (cp32020) - PUCRS 2026/2
 *
 * Mesmo problema do T1 (OpenMP). Um unico arquivo gera as DUAS versoes:
 *
 * gcc -O2 -o mandelbrot_seq mandelbrot.c -> sequencial
 * mpicc -O2 -DPARALELO -o mandelbrot_mpi mandelbrot.c -> MPI coord/trab
 *
 * Mapeamento para o modelo do enunciado (Figura 1):
 *
 * saco de trabalho = a imagem, uma matriz saco[altura][largura] de int
 * unidade (vetor) = uma LINHA da imagem (largura pixels); com -k agrupa
 * k linhas consecutivas numa mesma tarefa
 * processamento = calcular as iteracoes de Mandelbrot de cada pixel
 *
 * O coordenador (rank 0) nao calcula: so gerencia o saco. Os trabalhadores
 * (ranks 1..np-1) tomam a iniciativa, pedindo trabalho; recebem o indice da
 * proxima tarefa, calculam o vetor e devolvem o resultado, que ja serve como
 * o pedido seguinte. O coordenador recebe de MPI_ANY_SOURCE, ou seja, por
 * ordem de chegada, e guarda cada vetor na sua linha original do saco, o que
 * mantem a ordem da matriz independente de quem terminou primeiro. Quando o
 * saco esvazia, responde cada novo pedido com uma mensagem de termino.
 *
 * Saida no stdout, uma linha CSV:
 * modo;processos;linhas_por_tarefa;largura;altura;max_iter;tempo;checksum
 * O checksum (soma de todas as iteracoes) deve ser IDENTICO entre a versao
 * sequencial e a paralela: e a prova de corretude.
 *
 * Instrumentacao para a analise (nao altera o calculo; so mede):
 * -b carga de cada processo no stderr (ver imprime_carga)
 * -r arq linha do tempo de cada tarefa (rank;linha;inicio;fim), para o Gantt
 * -l arq (so no sequencial) custo de cada linha em iteracoes
 */

/* Habilita clock_gettime/getopt mesmo sob -std=c11 estrito. */
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

#ifdef PARALELO
#include <mpi.h>
#endif

/* Janela do plano complexo, igual ao T1. Fixa em todos os experimentos. */
#define RE_MIN -2.0
#define RE_MAX 0.6
#define IM_MIN -1.3
#define IM_MAX 1.3

/* Rank do coordenador e as etiquetas (tags) das mensagens do protocolo. */
#define COORDENADOR 0
#define TAG_PEDIDO 1 /* trab -> coord: "estou livre", sem dados */
#define TAG_RESULTADO 2 /* trab -> coord: vetor calculado (e novo pedido) */
#define TAG_TAREFA 3 /* coord -> trab: indice da primeira linha */
#define TAG_FIM 4 /* coord -> trab: saco vazio, pode terminar */

/* Parametros do problema, lidos da linha de comando. */
typedef struct {
    int largura;
    int altura;
    int max_iter;
    int linhas_por_tarefa; /* granularidade da unidade de trabalho */
    int mostrar_carga; /* -b */
    const char *arq_rastro; /* -r: linha do tempo das tarefas */
    const char *arq_perfil; /* -l: custo por linha (sequencial) */
} Parametros;

/*
 * Nucleo do calculo: itera z = z^2 + c e devolve em quantos passos o ponto
 * escapou do disco de raio 2, ou max_iter se nunca escapou. Pontos dentro do
 * conjunto custam max_iter iteracoes, pontos de fora escapam em poucas: o
 * custo de cada linha varia muito e nao da para prever, e por isso o saco de
 * trabalho com distribuicao sob demanda faz diferenca.
 */
static inline int mandel_ponto(double cr, double ci, int max_iter)
{
    double zr = 0.0, zi = 0.0;
    int it = 0;

    while (it < max_iter && zr * zr + zi * zi <= 4.0) {
        double zr_novo = zr * zr - zi * zi + cr;
        zi = 2.0 * zr * zi + ci;
        zr = zr_novo;
        it++;
    }
    return it;
}

/* Processa um vetor: calcula a linha y inteira e grava em vetor[0..largura). */
static void processa_linha(int y, int *vetor, const Parametros *p)
{
    const double passo_re = (RE_MAX - RE_MIN) / p->largura;
    const double passo_im = (IM_MAX - IM_MIN) / p->altura;
    const double ci = IM_MIN + y * passo_im;

    for (int x = 0; x < p->largura; x++)
        vetor[x] = mandel_ponto(RE_MIN + x * passo_re, ci, p->max_iter);
}

static long long soma_saco(const int *saco, const Parametros *p)
{
    long long soma = 0;
    for (size_t i = 0; i < (size_t)p->largura * p->altura; i++)
```

```
        soma += saco[i];
    return soma;
}

static int le_parametros(int argc, char **argv, Parametros *p)
{
    p->largura = 1024;
    p->altura = 1024;
    p->max_iter = 1000;
    p->linhas_por_tarefa = 1;
    p->mostrar_carga = 0;
    p->arq_rastro = NULL;
    p->arq_perfil = NULL;

    int opt;
    while ((opt = getopt(argc, argv, "w:a:i:k:br:l:")) != -1) {
        switch (opt) {
        case 'w': p->largura = atoi(optarg); break;
        case 'a': p->altura = atoi(optarg); break;
        case 'i': p->max_iter = atoi(optarg); break;
        case 'k': p->linhas_por_tarefa = atoi(optarg); break;
        case 'b': p->mostrar_carga = 1; break;
        case 'r': p->arq_rastro = optarg; break;
        case 'l': p->arq_perfil = optarg; break;
        default: return -1;
        }
    }
    if (p->largura <= 0 || p->altura <= 0 || p->max_iter <= 0 ||
        p->linhas_por_tarefa <= 0)
        return -1;
    return 0;
}

#ifdef PARALELO

/* Medidas de um processo durante a execucao, em segundos desde o inicio. */
typedef struct {
    double tarefas; /* quantas tarefas recebeu */
    double linhas; /* quantas linhas (vetores) processou */
    double ocupado; /* tempo calculando */
    double espera; /* tempo bloqueado em MPI_Recv, esperando o outro lado */
    double envio; /* tempo dentro de MPI_Send devolvendo resultados */
    double fim; /* instante em que recebeu FIM (trab) ou terminou (coord) */
} Carga;
#define CAMPOS_CARGA 6

/* Quantas linhas a tarefa que comeca em 'inicio' tem (a ultima pode ser menor). */
static int linhas_da_tarefa(int inicio, const Parametros *p)
{
    int resto = p->altura - inicio;
    return resto < p->linhas_por_tarefa ? resto : p->linhas_por_tarefa;
}

/*
 * Coordenador: gerencia o saco de trabalho.
 *
 * Mensagem de resultado = [indice_da_primeira_linha, pixels...]. O indice vai
 * junto do vetor para o coordenador saber onde guarda-lo no saco: como a
 * recepcao e por ordem de chegada, os vetores chegam fora de ordem.
 */
static void coordenador(int num_procs, const Parametros *p, int *saco,
                        double t0, Carga *c)
{
    const int tam_tarefa = p->largura * p->linhas_por_tarefa;
    int *buffer = malloc((size_t)(tam_tarefa + 1) * sizeof(int));

    int proxima_linha = 0; /* topo do saco: proxima a entregar */
    int ativos = num_procs - 1; /* trabalhadores ainda nao dispensados */

    while (ativos > 0) {
        MPI_Status st;

        /* Ordem de chegada: aceita pedido ou resultado de qualquer um. O tempo
         * parado aqui e o coordenador OCIOSO; se for alto, ele nao e gargalo. */
        double t_espera = MPI_Wtime();
        MPI_Recv(buffer, tam_tarefa + 1, MPI_INT, MPI_ANY_SOURCE,
                 MPI_ANY_TAG, MPI_COMM_WORLD, &st);
        c->espera += MPI_Wtime() - t_espera;
        int trabalhador = st.MPI_SOURCE;

        if (st.MPI_TAG == TAG_RESULTADO) {
            /* Devolve o vetor processado para a sua posicao original. */
            int inicio = buffer[0];
            int n = linhas_da_tarefa(inicio, p);
            memcpy(&saco[(size_t)inicio * p->largura], &buffer[1],
                   (size_t)n * p->largura * sizeof(int));
            c->linhas += n;
        }

        /* Todo pedido ou resultado e tambem um pedido de mais trabalho. */
        if (proxima_linha < p->altura) {
            MPI_Send(&proxima_linha, 1, MPI_INT, trabalhador, TAG_TAREFA,
                     MPI_COMM_WORLD);
            proxima_linha += p->linhas_por_tarefa;
            c->tarefas++;
        } else {
            MPI_Send(NULL, 0, MPI_INT, trabalhador, TAG_FIM, MPI_COMM_WORLD);
            ativos--;
        }
    }
    c->fim = MPI_Wtime() - t0;
    free(buffer);
}

/*
 * Trabalhador: pede trabalho, processa, devolve. A iniciativa e sempre dele;
 * o envio do resultado ja conta como o proximo pedido, economizando uma
 * mensagem por tarefa.
 *
 * rastro, se nao for NULL, recebe (linha, inicio, fim) de cada tarefa.
 */
static void trabalhador(const Parametros *p, double t0, Carga *c,
```

```
                    double *rastro)
{
    const int tam_tarefa = p->largura * p->linhas_por_tarefa;
    int *buffer = malloc((size_t)(tam_tarefa + 1) * sizeof(int));

    MPI_Send(NULL, 0, MPI_INT, COORDENADOR, TAG_PEDIDO, MPI_COMM_WORLD);

    for (;;) {
        MPI_Status st;
        int inicio;

        /* Tempo ate a tarefa chegar: latencia da rede + fila no coordenador. */
        double t_espera = MPI_Wtime();
        MPI_Recv(&inicio, 1, MPI_INT, COORDENADOR, MPI_ANY_TAG,
                 MPI_COMM_WORLD, &st);
        double t_chegou = MPI_Wtime();
        c->espera += t_chegou - t_espera;
        if (st.MPI_TAG == TAG_FIM) {
            c->fim = t_chegou - t0;
            break;
        }

        int n = linhas_da_tarefa(inicio, p);
        buffer[0] = inicio;
        for (int i = 0; i < n; i++)
            processa_linha(inicio + i, &buffer[1 + i * p->largura], p);
        double t_calculou = MPI_Wtime();
        c->ocupado += t_calculou - t_chegou;

        if (rastro) {
            double *r = &rastro[3 * (size_t)c->tarefas];
            r[0] = inicio;
            r[1] = t_chegou - t0;
            r[2] = t_calculou - t0;
        }
        c->tarefas++;
        c->linhas += n;

        MPI_Send(buffer, 1 + n * p->largura, MPI_INT, COORDENADOR,
                 TAG_RESULTADO, MPI_COMM_WORLD);
        c->envio += MPI_Wtime() - t_calculou;
    }
    free(buffer);
}

/*
 * Carga de todos os processos, no stderr, uma linha por rank:
 * carga;processos;k;rank;tarefas;linhas;ocupado;espera;envio;fim;tempo_total
 * Para o coordenador (rank 0) "tarefas" e "linhas" sao as entregues/recebidas.
 */
static void imprime_carga(const Carga *cargas, int num_procs,
                          const Parametros *p, double tempo)
{
    for (int r = 0; r < num_procs; r++) {
        const Carga *c = &cargas[r];
        fprintf(stderr, "carga;%d;%d;%d;%.0f;%.0f;%.6f;%.6f;%.6f;%.6f;%.6f\n",
                num_procs, p->linhas_por_tarefa, r, c->tarefas, c->linhas,
                c->ocupado, c->espera, c->envio, c->fim, tempo);
    }
}

/*
 * Junta no coordenador a linha do tempo de todos os trabalhadores e grava
 * rank;linha;inicio;fim. Cada trabalhador fez um numero diferente de tarefas
 * (e o ponto do saco de trabalho), entao o tamanho vem da carga de cada um.
 */
static void grava_rastro(const Parametros *p, int rank, int num_procs,
                         const Carga *minha, const Carga *cargas,
                         const double *rastro)
{
    int *tam = NULL, *desl = NULL;
    double *todos = NULL;
    int meu_tam = 3 * (int)minha->tarefas;
    if (rank == COORDENADOR)
        meu_tam = 0; /* o coordenador nao executa tarefas */

    if (rank == COORDENADOR) {
        tam = malloc(num_procs * sizeof(int));
        desl = malloc(num_procs * sizeof(int));
        int total = 0;
        for (int r = 0; r < num_procs; r++) {
            tam[r] = r == COORDENADOR ? 0 : 3 * (int)cargas[r].tarefas;
            desl[r] = total;
            total += tam[r];
        }
        todos = malloc((size_t)(total > 0 ? total : 1) * sizeof(double));
    }
    MPI_Gatherv(rastro, meu_tam, MPI_DOUBLE, todos, tam, desl, MPI_DOUBLE,
                COORDENADOR, MPI_COMM_WORLD);

    if (rank == COORDENADOR) {
        FILE *f = fopen(p->arq_rastro, "w");
        if (f) {
            fprintf(f, "rank;linha;inicio;fim\n");
            for (int r = 1; r < num_procs; r++)
                for (int i = 0; i < tam[r]; i += 3) {
                    const double *t = &todos[desl[r] + i];
                    fprintf(f, "%d;%.0f;%.6f;%.6f\n", r, t[0], t[1], t[2]);
                }
            fclose(f);
        } else {
            perror(p->arq_rastro);
        }
        free(tam);
        free(desl);
        free(todos);
    }
}

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
```

```
334
335     int rank, num_procs;
336     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
337     MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
338
339     Parametros p;
340     if (le_parametros(argc, argv, &p) != 0 || num_procs < 2) {
341         if (rank == COORDENADOR)
342             fprintf(stderr, "uso: mpirun -np N (N>=2) %s [-w largura] "
343                     "[-a altura] [-i max_iter] [-k linhas_por_tarefa] [-b] "
344                     "[-r rastro.csv]\n", argv[0]);
345         MPI_Finalize();
346         return 1;
347     }
348
349     int *saco = NULL;
350     if (rank == COORDENADOR) {
351         saco = malloc((size_t)p.largura * p.altura * sizeof(int));
352         if (!saco) {
353             fprintf(stderr, "memoria insuficiente para %dx%d\n",
354                     p.largura, p.altura);
355             MPI_Abort(MPI_COMM_WORLD, 1);
356         }
357     }
358
359     /* Espaco para a linha do tempo: no pior caso um trabalhador faz todas. */
360     double *rastro = NULL;
361     if (p.arq_rastro && rank != COORDENADOR) {
362         int max_tarefas = (p.altura + p.linhas_por_tarefa - 1)
363                           / p.linhas_por_tarefa;
364         rastro = malloc((size_t)max_tarefas * 3 * sizeof(double));
365     }
366
367     /* Todos partem juntos: o tempo mede so a distribuicao e o calculo. */
368     MPI_Barrier(MPI_COMM_WORLD);
369     double t0 = MPI_Wtime();
370
371     Carga minha = {0};
372     if (rank == COORDENADOR)
373         coordenador(num_procs, &p, saco, t0, &minha);
374     else
375         trabalhador(&p, t0, &minha, rastro);
376
377     double tempo = MPI_Wtime() - t0;
378
379     /* Daqui em diante, fora do tempo medido: coleta das medidas de analise. */
380     Carga *cargas = NULL;
381     if (rank == COORDENADOR)
382         cargas = malloc((size_t)num_procs * sizeof(Carga));
383     MPI_Gather(&minha, CAMPOS_CARGA, MPI_DOUBLE, cargas, CAMPOS_CARGA,
384                MPI_DOUBLE, COORDENADOR, MPI_COMM_WORLD);
385
386     if (p.arq_rastro)
387         grava_rastro(&p, rank, num_procs, &minha, cargas, rastro);
388
389     if (rank == COORDENADOR) {
390         printf("mpi;%d;%d;%d;%d;%d;%.6f;%lld\n", num_procs,
391                p.linhas_por_tarefa, p.largura, p.altura, p.max_iter, tempo,
392                soma_saco(saco, &p));
393         if (p.mostrar_carga)
394             imprime_carga(cargas, num_procs, &p, tempo);
395         free(cargas);
396         free(saco);
397     }
398     free(rastro);
399
400     MPI_Finalize();
401     return 0;
402 }
403
404 #else /* versao sequencial: o mesmo saco, processado vetor a vetor */
405
406 /* Relogio monotonico (MPI_Wtime nao existe sem MPI). */
407 static double agora_segundos(void)
408 {
409     struct timespec ts;
410     clock_gettime(CLOCK_MONOTONIC, &ts);
411     return ts.tv_sec + ts.tv_nsec * 1e-9;
412 }
413
414 /* Custo de cada linha (soma das iteracoes dos seus pixels): mostra de onde
415  * vem o desbalanceamento e permite prever a distribuicao estatica. */
416 static void grava_perfil(const int *saco, const Parametros *p)
417 {
418     FILE *f = fopen(p->arq_perfil, "w");
419     if (!f) {
420         perror(p->arq_perfil);
421         return;
422     }
423     fprintf(f, "linha;iteracoes\n");
424     for (int y = 0; y < p->altura; y++) {
425         long long soma = 0;
426         for (int x = 0; x < p->largura; x++)
427             soma += saco[(size_t)y * p->largura + x];
428         fprintf(f, "%d;%lld\n", y, soma);
429     }
430     fclose(f);
431 }
432
433 int main(int argc, char **argv)
434 {
435     Parametros p;
436     if (le_parametros(argc, argv, &p) != 0) {
437         fprintf(stderr, "uso: %s [-w largura] [-a altura] [-i max_iter] "
438                 "[-l perfil.csv]\n", argv[0]);
439         return 1;
440     }
441
442     int *saco = malloc((size_t)p.largura * p.altura * sizeof(int));
443     if (!saco) {
444         fprintf(stderr, "memoria insuficiente para %dx%d\n",
445                 p.largura, p.altura);
```

```
446         return 1;
447     }
448 
449     double t0 = agora_segundos();
450     for (int y = 0; y < p.altura; y++)
451         processa_linha(y, &saco[(size_t)y * p.largura], &p);
452     double tempo = agora_segundos() - t0;
453 
454     printf("seq;1;1;%d;%d;%d;%.6f;%lld\n", p.largura, p.altura, p.max_iter,
455            tempo, soma_saco(saco, &p));
456     if (p.arq_perfil)
457         grava_perfil(saco, &p);
458     free(saco);
459     return 0;
460 }
461 
462 #endif
```

Medição — `medir.sh` e `job.sh`

```
#!/usr/bin/env bash
# Coleta todos os dados do T2 em resultados/ (ou em $RES). Deve rodar dentro da alocacao de
# 2 nos exclusivos do job.sh. Experimentos:
#
# forte.csv imagem fixa, p = 2..32 -> speed-up forte
# fraca.csv pixels por processo fixos -> speed-up fraco
# carga.csv medidas de cada rank, para cada p -> decomposicao da eficiencia
# grao.csv linhas por tarefa (k) com p maximo -> granularidade/balanceamento
# intensidade.csv max_iter variando com p maximo -> computacao x comunicacao
# local.csv mesmo p em 1 no x 2 nos -> custo da rede x HT
# perfil.csv custo de cada linha da imagem -> origem do desbalanceamento
# rastro_*.csv linha do tempo de cada tarefa -> Gantt
#
# Cada tempo e a MEDIANA de REPS execucoes; o CSV guarda tambem min e max para
# as barras de variabilidade.
set -euo pipefail
cd "$(dirname "$0")"

PROCS=${PROCS:-"2 4 8 16 32"}
REPS=${REPS:-5}
ITER=${ITER:-2000}
LADO=${LADO:-4096} # imagem da escalabilidade forte (mesma do T1)
LADO_BASE=${LADO_BASE:-1024} # lado por processo na escalabilidade fraca
GRAOS=${GRAOS:-"1 2 4 8 16 64"}
LADO_INT=${LADO_INT:-2048} # imagem do experimento de intensidade
ITERS_INT=${ITERS_INT:-"250 500 1000 2000 4000 8000"}
PROCS_LOCAL=${PROCS_LOCAL:-"8 16"}
LOCAL=${LOCAL:-0} # 1 = teste no notebook, sem Slurm
RES=${RES:-resultados} # diretorio de saida

maior=$(echo $PROCS | tr ' ' '\n' | sort -n | tail -1)
estatico=$(( (LADO + maior - 2) / (maior - 1) )) # 1 bloco por trabalhador

# ---------------------------------------------------------- lancador -----
# Nucleos FISICOS por no: acima disso por no, os ranks passam a dividir nucleo
# com hyper-threading e o Open MPI precisa contar as threads HT como slots.
if [ "$LOCAL" = 1 ]; then
    NOS=(localhost); NUC=64
else
    NOS=($(scontrol show hostnames "$SLURM_JOB_NODELIST"))
    NUC=$(LC_ALL=C lscpu | awk -F: '/^Core\(s\) per socket/{c=$2} /^Socket\(s\)/{s=$2} END{print c*s}')
fi

# lanca NP ONDE programa args... ONDE = 2nos (round-robin) ou 1no
# Se MPIRUN estiver definido no ambiente, ele e usado como esta (ex.: srun).
lanca() {
    local np=$1 onde=$2; shift 2
    if [ -n "${MPIRUN:-}" ]; then $MPIRUN -n "$np" "$@"; return; fi
    if [ "$LOCAL" = 1 ]; then mpirun --oversubscribe -n "$np" "$@"; return; fi

    local por_no=$np ht=""
    [ "$onde" = 2nos ] && por_no=$(( (np + 1) / 2 ))
    [ "$por_no" -gt "$NUC" ] && ht="--use-hwthread-cpus"
    if [ "$onde" = 2nos ]; then
        mpirun --oversubscribe --map-by node $ht -n "$np" "$@"
    else
        mpirun --oversubscribe --host "${NOS[0]}:$((2 * NUC))" $ht -n "$np" "$@"
    fi
}

# Roda REPS vezes e imprime a linha da mediana + ;tmin;tmax.
mediana() {
    local i
    for ((i = 0; i < REPS; i++)); do "$@"; done | sort -t';' -k7 -g |
        awk -F";" -v n="$REPS" '{l[NR]=$0; t[NR]=$7}
            END {print l[int((n+1)/2)] ";" t[1] ";" t[n]}'
}

CAB='modo;processos;linhas_por_tarefa;largura;altura;max_iter;tempo;checksum;tmin;tmax'
mkdir -p "$RES"
tempo_de() { tail -1 "$1" | awk -F';' '{print $(NF-3)}'; } # coluna tempo

echo "nos: ${NOS[*]} | nucleos fisicos/no: $NUC | p: $PROCS | reps: $REPS"
echo ">> onde cai cada rank com p=$maior (2 nos) e p=16 (1 no):"
lanca "$maior" 2nos hostname | sort | uniq -c
lanca 16 1no hostname | sort | uniq -c

# ------------------------------------------------------- forte+carga -----
echo "$CAB" > $RES/forte.csv
echo "processos;k;rank;tarefas;linhas;ocupado;espera;envio;fim;tempo" > $RES/carga.csv
echo ">> forte ${LADO}x${LADO}, ${ITER} iter"
mediana ./mandelbrot_seq -w "$LADO" -a "$LADO" -i "$ITER" >> $RES/forte.csv
echo " seq $(tempo_de $RES/forte.csv) s"
./mandelbrot_seq -w "$LADO" -a "$LADO" -i "$ITER" -l $RES/perfil.csv > /dev/null

# Uma execucao extra com -b: e dela que sai a decomposicao da eficiencia.
carga() { # np k [args extras]
    local np=$1 k=$2; shift 2
    lanca "$np" 2nos ./mandelbrot_mpi -w "$LADO" -a "$LADO" -i "$ITER" -k "$k" -b "$@" \
        2>&1 >/dev/null | awk '/^carga;/{sub(/^carga;/,""); print}' >> $RES/carga.csv
}

for np in $PROCS; do
    mediana lanca "$np" 2nos ./mandelbrot_mpi -w "$LADO" -a "$LADO" -i "$ITER" \
        >> $RES/forte.csv
    echo " p=$np $(tempo_de $RES/forte.csv) s"
done

# ------------------------------------------------------------ grao -----
# O ultimo k da um unico bloco a cada trabalhador: distribuicao ESTATICA, sem
# saco de trabalho. E o contraste que mostra o ganho do balanceamento.
echo "$CAB" > $RES/grao.csv
echo ">> grao com p=$maior"
for k in $GRAOS $estatico; do
    mediana lanca "$maior" 2nos ./mandelbrot_mpi -w "$LADO" -a "$LADO" -i "$ITER" -k "$k" \
        >> $RES/grao.csv
    echo " k=$k $(tempo_de $RES/grao.csv) s"
done

```

```
# ------------------------------------------------------------- carga -----
# Por p com k=1 (decomposicao da eficiencia) e por k com p maximo (grao).
echo ">> carga por rank"
for np in $PROCS; do carga "$np" 1; done
for k in $GRAOS $estatico; do [ "$k" != 1 ] && carga "$maior" "$k"; done
# Linha do tempo (Gantt): saco de trabalho x um bloco fixo por trabalhador.
for k in 1 "$estatico"; do
    lanca "$maior" 2nos ./mandelbrot_mpi -w "$LADO" -a "$LADO" -i "$ITER" -k "$k" \
        -r "$RES/rastro_p${maior}_k${k}.csv" > /dev/null
done

# ------------------------------------------------------------- fraca -----
# Pixels crescem por p (lado por sqrt(p)) sobre a mesma janela do plano: o
# custo medio por pixel fica constante.
echo "$CAB" > $RES/fraca.csv
echo ">> fraca ${LADO_BASE}x${LADO_BASE} por processo"
mediana ./mandelbrot_seq -w "$LADO_BASE" -a "$LADO_BASE" -i "$ITER" >> $RES/fraca.csv
for np in $PROCS; do
    lado=$(awk -v b="$LADO_BASE" -v p="$np" 'BEGIN{printf "%d", int(b*sqrt(p)+0.5)}')
    mediana lanca "$np" 2nos ./mandelbrot_mpi -w "$lado" -a "$lado" -i "$ITER" \
        >> $RES/fraca.csv
    echo " p=$np ${lado}x${lado} $(tempo_de $RES/fraca.csv) s"
done

# ---------------------------------------------------------- intensidade -----
# Mesmo numero de mensagens e bytes, computacao por mensagem variando com
# max_iter: mostra a partir de que ponto a comunicacao domina.
echo "$CAB" > $RES/intensidade.csv
echo ">> intensidade ${LADO_INT}x${LADO_INT}, p=$maior"
for it in $ITERS_INT; do
    mediana ./mandelbrot_seq -w "$LADO_INT" -a "$LADO_INT" -i "$it" >> $RES/intensidade.csv
    mediana lanca "$maior" 2nos ./mandelbrot_mpi -w "$LADO_INT" -a "$LADO_INT" -i "$it" \
        >> $RES/intensidade.csv
    echo " iter=$it $(tempo_de $RES/intensidade.csv) s"
done

# ------------------------------------------------------------- local -----
# p=8: 1 no (1 rank/nucleo) x 2 nos (4+4) -> custo so da rede
# p=16: 1 no (2 ranks/nucleo, com HT) x 2 nos (8+8) -> HT x rede
echo "onde;$CAB" > $RES/local.csv
echo ">> 1 no x 2 nos"
for np in $PROCS_LOCAL; do
    for onde in 1no 2nos; do
        linha=$(mediana lanca "$np" "$onde" ./mandelbrot_mpi -w "$LADO" -a "$LADO" -i "$ITER")
        echo "$onde;$linha" >> $RES/local.csv
        echo " p=$np $onde $(tempo_de $RES/local.csv) s"
    done
done

echo "pronto. CSVs em $RES/"
```

```
#!/usr/bin/env bash
#SBATCH --job-name=mandel-mpi-g20
#SBATCH --nodes=2
#SBATCH --exclusive
#SBATCH --time=02:00:00
#SBATCH --output=slurm-%j.out
#
# 2 nos exclusivos da atlantica (8 nucleos / 16 threads HT cada), como pede o
# enunciado. --exclusive garante que ninguem divide os nos com a medicao.
# A bateria completa (medir.sh) leva em torno de 30 min.
# Se o sbatch reclamar de particao, veja "sinfo" e acrescente --partition.
#
# submeter: sbatch job.sh acompanhar: squeue -u $USER ; tail -f slurm-*.out
set -euo pipefail
cd "$SLURM_SUBMIT_DIR"

mkdir -p resultados
{ echo "nos: $SLURM_JOB_NODELIST"; LC_ALL=C lscpu; free -g; gcc --version | head -1;
  mpirun --version 2>&1 | head -1; } > resultados/maquina.txt 2>&1

make clean && make
./medir.sh
python3 analisar.py | tee resultados/tabelas.txt

# matplotlib pode nao existir no no; nesse caso gere os graficos localmente.
if python3 -c "import matplotlib" 2>/dev/null; then
    python3 graficos.py
else
    echo "matplotlib ausente: rode 'python3 graficos.py' na maquina local"
fi
```

Análise — `analisar.py`

```python
#!/usr/bin/env python3
"""Le os CSVs de medir.sh e imprime as tabelas da analise do T2.

    python3 analisar.py [diretorio_de_resultados]

So usa a biblioteca padrao, para rodar no proprio no do cluster. As funcoes de
calculo sao reaproveitadas por graficos.py, para que tabela e grafico saiam
sempre dos mesmos numeros.

Convencoes (p conta TODOS os processos, inclusive o coordenador):
    forte: S(p) = Tseq / T(p)  E(p) = S(p) / p
    fraca: S(p) = p * Tseq / T(p)  E(p) = Tseq / T(p)
"""
import csv
import os
import statistics
import sys

DIR = sys.argv[1] if len(sys.argv) > 1 else "resultados"


# ------------------------------------------------------------------ leitura --
def le(nome):
    caminho = os.path.join(DIR, nome)
    if not os.path.exists(caminho):
        return None
    with open(caminho) as f:
        return list(csv.DictReader(f, delimiter=";"))


def execucoes(nome):
    """(linha_seq, [linhas_mpi]) com os campos numericos convertidos."""
    linhas = le(nome) or []
    for r in linhas:
        for c in ("processos", "linhas_por_tarefa", "largura", "altura",
                  "max_iter"):
            r[c] = int(r[c])
        for c in ("tempo", "tmin", "tmax"):
            r[c] = float(r[c])
    seq = [r for r in linhas if r["modo"] == "seq"]
    mpi = [r for r in linhas if r["modo"] == "mpi"]
    return seq, mpi


def cargas():
    """{(p, k): {'coord': linha_rank0, 'trab': [linhas dos trabalhadores]}}"""
    grupos = {}
    for r in le("carga.csv") or []:
        p, k, rank = int(r["processos"]), int(r["k"]), int(r["rank"])
        reg = {c: float(r[c]) for c in ("tarefas", "linhas", "ocupado",
                                        "espera", "envio", "fim", "tempo")}
        reg["rank"] = rank
        g = grupos.setdefault((p, k), {"coord": None, "trab": []})
        if rank == 0:
            g["coord"] = reg
        else:
            g["trab"].append(reg)
    return grupos


# ------------------------------------------------------------------ calculo --
def escala(nome, fraca):
    """Pontos [(p, T, S, E, S_min, S_max)] de forte.csv ou fraca.csv.
    S_min/S_max vem de tmax/tmin: a faixa de variacao entre as repeticoes."""
    seq, mpi = execucoes(nome)
    if not seq:
        return None, []
    tseq = seq[0]["tempo"]
    pts = []
    for r in sorted(mpi, key=lambda r: r["processos"]):
        p = r["processos"]
        num = p * tseq if fraca else tseq
        s = num / r["tempo"]
        e = (tseq / r["tempo"]) if fraca else s / p
        pts.append((p, r["tempo"], s, e, num / r["tmax"], num / r["tmin"]))
    return seq[0], pts


PARCELAS = ("util", "inflacao", "coordenador", "comunicacao", "cauda", "outros")


def decomposicao(g, tseq):
    """Divide os recursos gastos, p * T, em parcelas que somam 1.

    util Tseq: o trabalho que o sequencial tambem faria
    inflacao soma(ocupado) - Tseq: o mesmo calculo ficou mais lento em
               paralelo (HT dividindo o nucleo, cache, frequencia)
    coordenador T inteiro do rank 0, que nao calcula
    comunicacao soma do tempo dos trabalhadores dentro do MPI: esperando
               tarefa (Recv) e devolvendo resultado (Send)
    cauda soma de (T - fim_i): trabalhador ja dispensado esperando o
               ultimo terminar -> desbalanceamento no final
    outros o resto: laco, copias, medicao e preempcao pelo SO
    A parcela 'util' e exatamente a eficiencia E = Tseq / (p T).
    """
    trab = g["trab"]
    p = len(trab) + 1
    t = g["coord"]["tempo"]
    total = p * t
    ocupado = sum(w["ocupado"] for w in trab)
    partes = {
        "util": tseq,
        "inflacao": ocupado - tseq,
        "coordenador": t,
        "comunicacao": sum(w["espera"] + w["envio"] for w in trab),
        "cauda": sum(t - w["fim"] for w in trab),
    }
    partes["outros"] = total - sum(partes.values())
    return {k: v / total for k, v in partes.items()}
```

```
110
111
112 def balanceamento(g):
113     """Metricas de equilibrio entre os trabalhadores de uma execucao."""
114     ocup = [w["ocupado"] for w in g["trab"]]
115     lin = [w["linhas"] for w in g["trab"]]
116     media = statistics.mean(ocup)
117     return {
118         "ocup_min": min(ocup), "ocup_max": max(ocup),
119         "max_med": max(ocup) / media, # 1.0 = perfeito
120         "cv": statistics.pstdev(ocup) / media, # dispersao relativa
121         "lin_min": min(lin), "lin_max": max(lin),
122         "coord_ocioso": g["coord"]["espera"] / g["coord"]["tempo"],
123     }
124
125
126 def perfil():
127     """Custo (iteracoes) de cada linha da imagem, na ordem."""
128     return [int(r["iteracoes"]) for r in le("perfil.csv") or []]
129
130
131 def previsao_estatica(custos, trabalhadores, tseq):
132     """Tempo previsto se cada trabalhador recebe um bloco contiguo fixo: o
133     mais caro dita o fim. Comparado ao medido, valida que o desbalanceamento
134     vem do custo das linhas e nao de outro efeito."""
135     bloco = -(-len(custos) // trabalhadores)
136     somas = [sum(custos[i:i + bloco]) for i in range(0, len(custos), bloco)]
137     return tseq * max(somas) / sum(custos), max(somas) / statistics.mean(somas)
138
139
140 # ------------------------------------------------------------------ tabelas --
141 def tabela_escala(titulo, nome, fraca):
142     seq, pts = escala(nome, fraca)
143     if not seq:
144         return
145     print("=" * 18, titulo, "=" * 18)
146     print("%-5s %-11s %9s %8s %9s %8s" % ("p", "imagem", "tempo(s)", "speedup",
147                                           "efic.", "faixa S"))
148     print("%-5s %-11s %9.3f" % ("seq", "%dx%d" % (seq["largura"], seq["altura"]),
149                                 seq["tempo"]))
150     _, mpi = execucoes(nome)
151     imagens = {r["processos"]: "%dx%d" % (r["largura"], r["altura"]) for r in mpi}
152     for p, t, s, e, smin, smax in pts:
153         print("%-5d %-11s %9.3f %8.2f %8.1f%% %4.2f-%.2f" %
154               (p, imagens[p], t, s, 100 * e, smin, smax))
155     # Corretude: na forte a imagem e a mesma, logo o checksum tambem.
156     if not fraca:
157         ruins = [r["processos"] for r in mpi if r["checksum"] != seq["checksum"]]
158         print("checksum:", "OK, identico ao sequencial (%s)" % seq["checksum"]
159               if not ruins else "DIFERE em p=%s" % ruins)
160     print()
161
162
163 def main():
164     tabela_escala("ESCALABILIDADE FORTE", "forte.csv", False)
165     tabela_escala("ESCALABILIDADE FRACA", "fraca.csv", True)
166
167     seq, _ = execucoes("forte.csv")
168     tseq = seq[0]["tempo"] if seq else None
169     grupos = cargas()
170
171     if grupos and tseq:
172         print("=" * 10, "DECOMPOSICAO DA EFICIENCIA (k=1, % de p*T)", "=" * 10)
173         print("%-4s" % "p" + "".join("%12s" % c for c in PARCELAS)
174               + "%10s" % "teto p-1")
175         for (p, k) in sorted(grupos):
176             if k != 1:
177                 continue
178             d = decomposicao(grupos[(p, k)], tseq)
179             print("%-4d" % p + "".join("%11.1f%%" % (100 * d[c]) for c in PARCELAS)
180                   + "%9.1f%%" % (100 * (p - 1) / p))
181         print(" util = eficiencia; teto p-1 = eficiencia maxima com 1 coordenador ocioso")
182         print()
183
184         print("=" * 14, "BALANCEAMENTO POR EXECUCAO", "=" * 14)
185         print("%-4s %-5s %9s %9s %8s %7s %8s %8s %10s" %
186               ("p", "k", "ocup.min", "ocup.max", "max/med", "CV", "lin.min",
187                "lin.max", "coord.ocio"))
188         for (p, k) in sorted(grupos):
189             b = balanceamento(grupos[(p, k)])
190             print("%-4d %-5d %9.3f %9.3f %8.3f %6.1f%% %8d %8d %9.1f%%" %
191                   (p, k, b["ocup_min"], b["ocup_max"], b["max_med"],
192                    100 * b["cv"], b["lin_min"], b["lin_max"],
193                    100 * b["coord_ocioso"]))
194         print()
195
196     _, grao = execucoes("grao.csv")
197     if grao:
198         melhor = min(r["tempo"] for r in grao)
199         p = grao[0]["processos"]
200         print("=" * 12, "GRANULARIDADE (p=%d, linhas por tarefa)" % p, "=" * 12)
201         print("%-6s %8s %9s %9s %10s" % ("k", "tarefas", "tempo(s)", "efic.",
202                                          "vs melhor"))
203         for r in grao:
204             k = r["linhas_por_tarefa"]
205             print("%-6d %8d %9.3f %8.1f%% %9.2fx" %
206                   (k, -(-r["altura"] // k), r["tempo"],
207                    100 * tseq / r["tempo"] / p if tseq else 0, r["tempo"] / melhor))
208         custos = perfil()
209         if custos and tseq:
210             k_est = max(r["linhas_por_tarefa"] for r in grao)
211             medido = max(grao, key=lambda r: r["linhas_por_tarefa"])["tempo"]
212             prev, razao = previsao_estatica(custos, p - 1, tseq)
213             print("estatico (k=%d): previsto pelo perfil %.3f s (bloco mais caro"
214                   " = %.2fx a media), medido %.3f s" % (k_est, prev, razao, medido))
215             print("saco ideal: Tseq/(p-1) = %.3f s" % (tseq / (p - 1)))
216         print()
217
218     _, inten = execucoes("intensidade.csv")
219     seq_i, _ = execucoes("intensidade.csv")
220     if inten:
221         tseq_it = {r["max_iter"]: r["tempo"] for r in seq_i}
```

```
        print("=" * 12, "INTENSIDADE (computacao por mensagem)", "=" * 12)
        print("%-8s %10s %10s %8s %8s %16s" % ("max_iter", "seq(s)", "mpi(s)",
                                               "speedup", "efic.", "us calc/msg"))
        for r in inten:
            it, p = r["max_iter"], r["processos"]
            s = tseq_it[it] / r["tempo"]
            # tempo de calculo sequencial por linha = trabalho entregue por mensagem
            us = 1e6 * tseq_it[it] / r["altura"]
            print("%-8d %10.3f %10.3f %8.2f %7.1f%% %16.0f" %
                  (it, tseq_it[it], r["tempo"], s, 100 * s / p, us))
        print()

    local = le("local.csv")
    if local and tseq:
        print("=" * 16, "1 NO x 2 NOS", "=" * 16)
        print("%-4s %-6s %9s %8s %9s" % ("p", "onde", "tempo(s)", "speedup", "efic."))
        for r in local:
            p, t = int(r["processos"]), float(r["tempo"])
            print("%-4d %-6s %9.3f %8.2f %8.1f%%" % (p, r["onde"], t, tseq / t,
                                                     100 * tseq / t / p))
        print()


# ------------------------------------------------------------------- latex --
def br(x, casas):
    """Numero no formato do relatorio: virgula decimal."""
    return ("%.*f" % (casas, x)).replace(".", ",")


def grava(nome, linhas):
    with open(os.path.join(DIR, nome), "w") as f:
        f.write("\n".join(linhas) + "\n")


def gera_latex():
    """Escreve as linhas das tabelas do relatorio em resultados/tab_*.tex.
    O relatorio.tex inclui esses arquivos quando existem: nenhum numero e
    copiado a mao para as tabelas."""
    seq, pts = escala("forte.csv", False)
    if not seq:
        return
    tseq = seq["tempo"]
    linhas = ["seq & %s & --- & --- & --- \\\\" % br(tseq, 3)]
    for p, t, s, e, _, _ in pts:
        linhas.append("%d & %s & %s & %s & %s \\\\" % (
            p, br(t, 3), br(s, 2), br(100 * e, 1), br(100 * (p - 1) / p, 1)))
    grava("tab_forte.tex", linhas)

    seq_f, pts_f = escala("fraca.csv", True)
    if seq_f:
        _, mpi = execucoes("fraca.csv")
        lado = {r["processos"]: r["largura"] for r in mpi}
        linhas = ["seq & %d$^2$ & %s & 1,00 & 100,0 \\\\" %
                  (seq_f["largura"], br(seq_f["tempo"], 3))]
        for p, t, s, e, _, _ in pts_f:
            linhas.append("%d & %d$^2$ & %s & %s & %s \\\\" % (
                p, lado[p], br(t, 3), br(s, 2), br(100 * e, 1)))
        grava("tab_fraca.tex", linhas)

    grupos = cargas()
    linhas = []
    for (p, k) in sorted(grupos):
        if k == 1:
            d = decomposicao(grupos[(p, k)], tseq)
            linhas.append("%d & " % p + " & ".join(
                br(100 * d[c], 1) for c in PARCELAS[:5]) + " \\\\")
    if linhas:
        grava("tab_decomposicao.tex", linhas)

    _, grao = execucoes("grao.csv")
    if grao:
        melhor = min(r["tempo"] for r in grao)
        p = grao[0]["processos"]
        linhas = []
        for r in grao:
            k = r["linhas_por_tarefa"]
            g = grupos.get((p, k))
            mm = br(balanceamento(g)["max_med"], 2) if g else "---"
            linhas.append("%d & %s & %s & %s & %s$\\times$ \\\\" % (
                k, br(r["tempo"], 3), br(100 * tseq / r["tempo"] / p, 1), mm,
                br(r["tempo"] / melhor, 2)))
        grava("tab_grao.tex", linhas)


if __name__ == "__main__":
    main()
    gera_latex()
```